Predicción de riesgo de fuga (Churn bancario)



Máster de Especialización en Inteligencia Artificial y Big Data

Neco Ruiz Crisóstomo

Resumen 2
Introducción 3
Objetivos 3
Marco teórico 4
Dataset 6
EDA (Análisis exploratorio) 8
Pipeline A 10
Pipeline A + Regresión Logística 12
Pipeline A + Random Forest 14
Pipeline B 17
Pipeline B + Regresión Logística 19
Pipeline B + Random Forest 22
Pipeline B + XgBoost 24
Modelo ensamblado: VotingClassifier 27
Pipeline B + VotingClassifier 28
Tuneado de Hiperparámetros para XGBOOST 31
Pipeline B + VotingClassifier + XGBoost optimizado 33
Elección del modelo 35
Conclusión 36
Bibliografía 37


DIGI
TECH
Centro Oficial FP
Digital & Tech



Resumen

El contexto en el que se desarrolla el proyecto es el de una empresa bancaria que muestra gran interés por detectar los riesgos de fuga de los clientes. Lo interesante del proyecto es ver cómo se comportan ciertos modelos de Machine Learning frente a dos Pipelines (uno básico y otro un poco más avanzado).

Estos modelos se van a enriquecer de un dataset que veremos posteriormente. El proyecto cuenta con un análisis exploratorio de los datos, limpieza, procesamiento de estos y técnicas de evaluación, así como su visualización.

Introducción

Este tipo de modelo (Chrun) surgió como herramienta analítica capaz de detectar fugas de los clientes con las empresas, gracias a modelos de Machine Learning. Esto supuso un gran avance para las compañías ya que, gracias a esta tecnología, se ahorraron grandes cantidades de dinero. De igual manera, mejoraron sus relaciones con los clientes, cuidando y ofreciendo mejores servicios para evitar estas desvinculaciones. Dicho esto, he aplicado todo lo aprendido durante el máster para intentar llevar a cabo una solución para esta empresa.

Objetivos

Como objetivos específicos, he planteado los siguientes:

- Capacidad para llevar a cabo un análisis exploratorio de los datos de manera óptima. Con la finalidad de limpiar el dataset, encontrar relaciones entre variables, detectar las más relevantes y eliminar aquellas que considere que no aportan información valiosa.
- Realizar un preprocessing exhaustivo, construyendo dos pipelines para posteriormente, estudiar cómo se comportan los modelos y ver por qué unos se comportan mejor que otros.
- Escoger el modelo más equilibrado e intentar mejorarlo.

Marco teórico

¿Qué es Machine Learning?

El machine learning es el subconjunto de la inteligencia artificial (IA) centrado en algoritmos que pueden "aprender" los patrones de los datos de entrenamiento y, posteriormente, hacer inferencias precisas sobre nuevos datos. Esta capacidad permite a los modelos tomar decisiones o realizar predicciones sin instrucciones explícitas y codificadas de forma rígida.

Tipos de aprendizaje

Durante el curso hemos visto 3 tipos de aprendizaje:

- **Aprendizaje supervisado:** Es aquel que trabaja con datos que previamente son etiquetados. Posteriormente el modelo, antes de la predicción ya conoce el resultado real.
- **Aprendizaje no supervisado:** Los modelos no conocen el resultado ya que no reciben datos etiquetados. Absorben estos de manera “cruda” para posteriormente encontrar patrones y relaciones entre ellos.
- **Aprendizaje por refuerzo:** Es aquel aprendizaje en el que un agente es capaza de tomar decisiones mediante prueba y error.

En este caso de uso, el aprendizaje a utilizar va a ser de tipo de supervisado con el objetivo de clasificar una variable categórica. Esto se debe a que se va a trabajar con datos etiquetados y, además, el veredicto del modelo va a ser una clasificación de una variable que indica que un cliente tiene o no riesgo de fuga.

Métricas de evaluación más usadas

- **Precision:** Esta mide cuántas veces el modelo acierta cuando predice un churn. Para mi caso, mediría de todos los clientes que el modelo detecta como posible abandono, ¿cuántos realmente abandonan? Esto es importante ya que una precesión baja, implica demasiados falsos positivos. Lo que desencadenaría costes adicionales, campañas innecesarias, etc...

- **Recall:** Bajo mi punto de vista, la métrica más importante ya que mide cuántos clientes que abandonan el banco son detectados.
- **F1-score:** Ha sido una de las métricas más importantes a la hora de comprar el comportamiento de los modelos. Combina precision y recall. Ha sido esencial ya que el dataset está desbalanceado, permitiendo accuracy's muy altos.
- **Curva ROC-AUC:** Para un proyecto de estas características, considero que es esencial saber si el modelo es capaz de separar bien ambas clases, por ello ha sido una de las más usadas.

Dataset

El dataset alberga una serie de variables que aportan información acerca de los clientes de un banco. Estas variables son la siguientes:

- RowNumber: Corresponde al número de registro o fila.
- CustomerId y Surname: Ambas contienen identificadores y apellidos de clientes respectivamente.
- CreditScore: Puntuaje crediticio.

- Geography: De dónde es el cliente.
- Gender: El género del cliente podría tener un papel en su decisión de churn.
- Age: Edad de los clientes.
- Tenure: Se refiere al número de años que el cliente ha estado con el banco.
- Balance: Saldo de los clientes.
- NumOfProducts: Indica el número de productos que un cliente ha adquirido a través del banco.
- HasCrCard: Denota si un cliente tiene una tarjeta de crédito. Indica si el cliente tiene tarjeta de crédito (1 = Sí, 0 = No).
- IsActiveMember: Indica si el cliente es un miembro activo (1 = Sí, 0 = No).
- EstimatedSalary: Salario de los clientes.
- Exited: Indica si el cliente abandonó el banco (1 = Sí, 0 = No). **Es la Variable objetivo**.
- Complain: Refleja si un cliente ha presentado una queja o no (1 = Sí, 0 = No).
- Satisfaction Score: Puntuación proporcionada por el cliente sobre la resolución de su queja.
- Card Type: Indica el tipo de tarjeta que posee el cliente.
- Points Earned: Refleja los puntos que el cliente ha ganado al usar su tarjeta de crédito.

El dataset ha sido extraído de Kaggle: “Customer-Churn-Records.csv”

EDA (Análisis exploratorio)

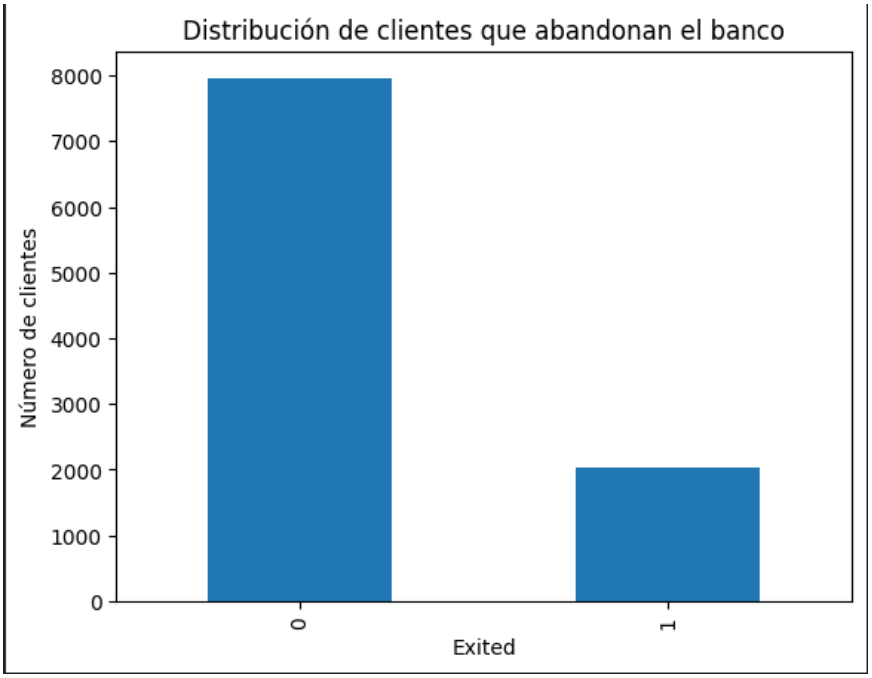
En primer lugar, analicé la calidad de los datos:

```
df.info()

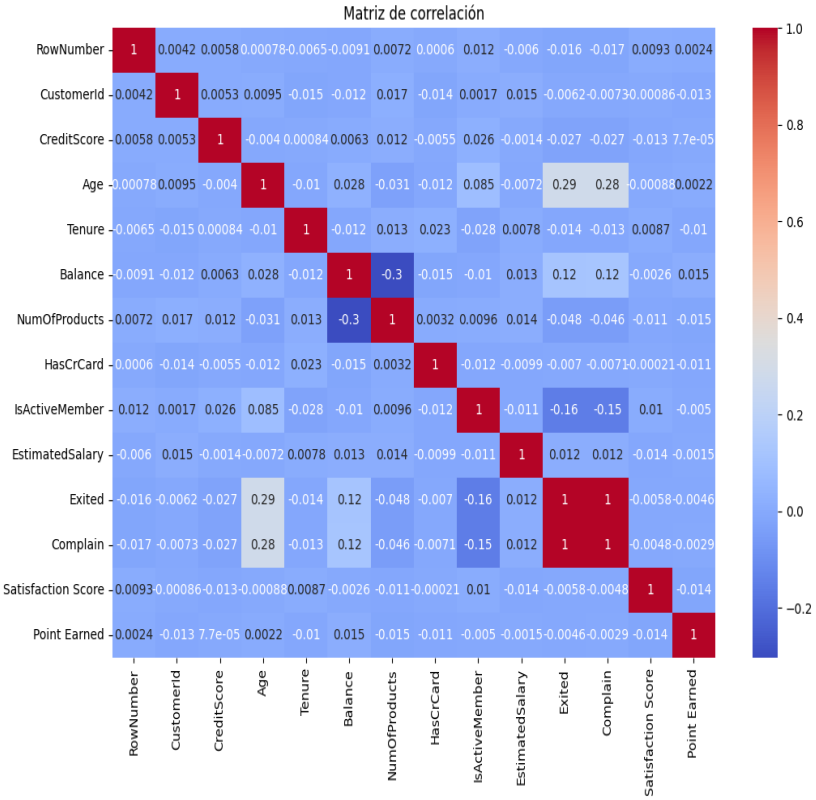
df.isnull().sum()

df.duplicated().sum()
```

Como resultados, el dataset no mostró registros duplicados ni nulos. Posteriormente, contemplé la distribución de las clases que influirán en la variable objetivo. Detecté una amenaza ya que hay una clase que es claramente la dominante (0, no abandonaron) ya que contiene más registros. Esto, influirá enormemente en el modelo ya que el dataset está desbalanceado.



A continuación, elaboré una matriz de correlaciónpara ver la relación que existía entre las variables. Observé que no había correlaciones muy elevadas, pero sí alguna que aportaba información redundante (todo esto será tratado en los pipelines o preprocessing).


Matriz de correlación
RowNumber 1 0.0042 0.0058 -0.00078 -0.0065 -0.0091 0.0072 0.0006 0.012 -0.006 -0.016 -0.017 0.0093 0.0024
CustomerId 0.0042 1 0.0053 0.0095 -0.015 -0.012 0.017 -0.014 0.0017 0.015 -0.0062 -0.0073 -0.00086 -0.013
CreditScore 0.0058 0.0053 1 -0.004 0.00084 0.0063 0.012 -0.0055 0.026 -0.0014 -0.027 -0.027 -0.013 7.7e-05
Age -0.00078 0.0095 -0.004 1 -0.01 0.028 -0.031 -0.012 0.085 -0.0072 0.29 0.28 -0.00088 0.0022
Tenure -0.0065 -0.015 0.00084 -0.01 1 -0.012 0.013 0.023 -0.028 0.0078 -0.014 -0.013 0.0087 -0.01
Balance -0.0091 -0.012 0.0063 0.028 -0.012 1 -0.3 -0.015 -0.01 0.013 0.12 0.12 -0.0026 0.015
NumOfProducts 0.0072 0.017 0.012 -0.031 0.013 -0.3 1 0.0032 0.0096 0.014 -0.048 -0.046 -0.011 -0.015
HasCrCard 0.0006 -0.014 -0.0055 -0.012 0.023 -0.015 0.0032 1 -0.012 -0.0099 -0.007 -0.0071 -0.00021 -0.011
IsActiveMember 0.012 0.0017 0.026 0.085 -0.028 -0.01 0.0096 -0.012 1 -0.011 -0.16 -0.15 0.01 -0.005
EstimatedSalary -0.006 0.015 -0.0014 -0.0072 0.0078 0.013 0.014 -0.0099 -0.011 1 0.012 0.012 -0.014 -0.0015
Exited -0.016 -0.0062 -0.027 0.29 -0.014 0.12 -0.048 -0.007 -0.16 0.012 1 1 -0.0058 -0.0046
Complain -0.017 -0.0073 -0.027 0.28 -0.013 0.12 -0.046 -0.0071 -0.15 0.012 1 1 -0.0048 -0.0029
Satisfaction Score 0.0093 -0.00086 -0.013 -0.00083 0.0087 -0.0026 -0.011 -0.00021 0.01 -0.014 -0.0058 -0.0048 1 -0.014
Point Earned 0.0024 -0.013 7.7e-05 0.0022 -0.01 0.015 -0.015 -0.011 -0.005 -0.0015 -0.0046 -0.0029 -0.014 1
RowNumber
CustomerId
CreditScore
Age
Tenure
Balance
NumOfProducts
HasCrCard
IsActiveMember
EstimatedSalary
Exited
Complain
Satisfaction Score
Point Earned
1.0
0.8
0.6
0.4
0.2
0.0
−0.2

Pipeline A

Para llevar a cabo el desarrollo de este “Baseline”, lo primero que hice fue eliminar las variables que aportaban el mismo valor. En este caso, “Exited” y “Complain” mostraban una fuerte relación de 1 y por ello la eliminé, así como otras que consideraba irrelevantes.

```
df = df.drop(["RowNumber", "CustomerId", "Surname", "Complain"], axis=1)
```

Posteriormente, separé la variable objetivo (Exited). Dividí el conjunto de los datos de entrenamiento y prueba en 80% train y 20% test (usando stratify=1 para mantener proporciones).

Para este primer Pipeline se llevó a cabo un desarrollo muy sencillo donde simplemente identifiqué las variables numéricas y se las imputó la media. A las variables categóricas, se las aplicó One-Hot Encoding para que fuesen representadas con valores numéricos y, además fueron imputadas con el valor más frecuente.

```
x = df.drop("Exited", axis=1)
y = df["Exited"]
```

```python
X_train, X_test, y_train, y_test = train_test_split(
    x,y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```python
num_transformer_A = Pipeline(steps=[("imputer", SimpleImputer(strategy="mean"))])

cat_transformer_A = Pipeline(steps=[("imputer", SimpleImputer(strategy="most_frequent")),
                                    ("encoder", OneHotEncoder(handle_unknown="ignore"))])
```

Pipeline A + Regresión Logística

Es un modelo de clasificación binaria, basado en una función cuya salida es una probabilidad entre 0 y 1 (transforma valores reales en probabilidades. Este modelo se ajusta perfectamente a las necesidades que requiere este caso de uso.

Modelado: Regresion Logistica

```python
modeloLR_pipeline_A = Pipeline([
    ("preprocessor", preprocessor_A),
    ("model", LogisticRegression(random_state=42, max_iter=5000, solver="liblinear"))
])
```

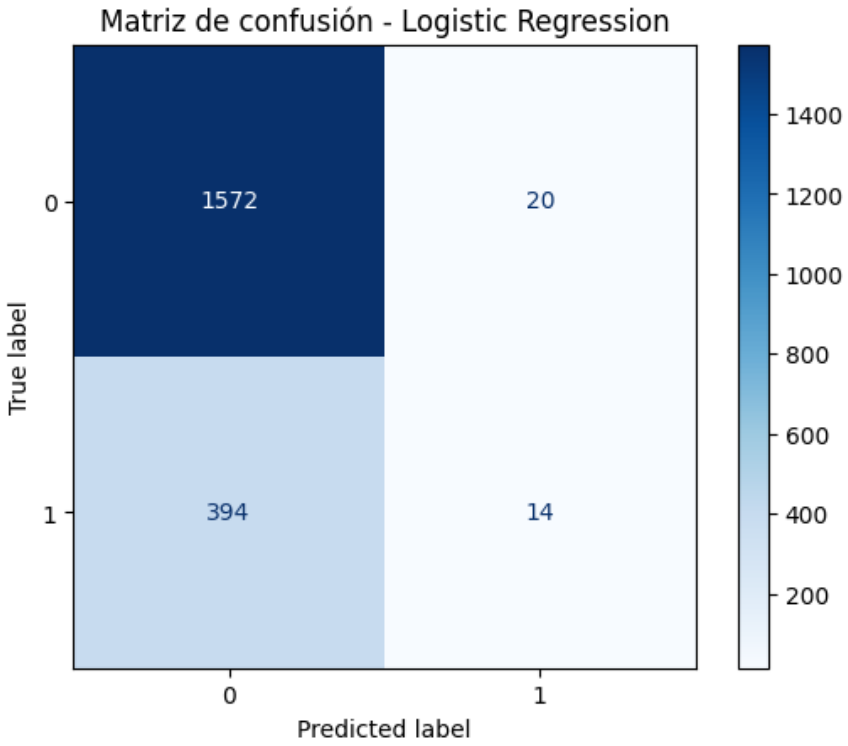
Generate

```python
modeloLR_pipeline_A.fit(X_train, y_train)
prediction_lr = modeloLR_pipeline_A.predict(X_test)
```

Resultados y visualización

Tras aplicar el Pipeline A junto con el modelo de regresión logística, se pude observar que se comporta muy por debajo de sus posibilidades. Alcanza una precisión del 79%, en cambio, otras métricas que enseñaré a continuación muestran que el modelo tiene limitaciones. La matriz de confusión muestra que es capaz de clasificar a los clientes que pertenecen a la clase mayoritaria (recall 99%), en cambio, no es capaz de identificar a esa parte de los clientes que pertenecen a la otra clase (3%). Esto yo creo que se debe al desbalanceo que existe entre las clases, además que no se ha tratado con ningún tipo de normalización. Por último, gracias a la curva ROC, observo que el modelo capta ciertos patrones, pero el valor que refleja sigue siendo insuficiente. (0.67)

	precision	recall	f1-score	support
0	0.80	0.99	0.88	1592
1	0.41	0.03	0.06	408
accuracy			0.79	2000
macro avg	0.61	0.51	0.47	2000
weighted avg	0.72	0.79	0.72	2000



Pipeline A + Random Forest

Es un modelo de aprendizaje supervisado que se caracteriza por forma de tratar a los datos. Se organiza en forma de múltiples árboles que colaboran de manera conjunta para buscar patrones y conseguir predecir de manera correcta.

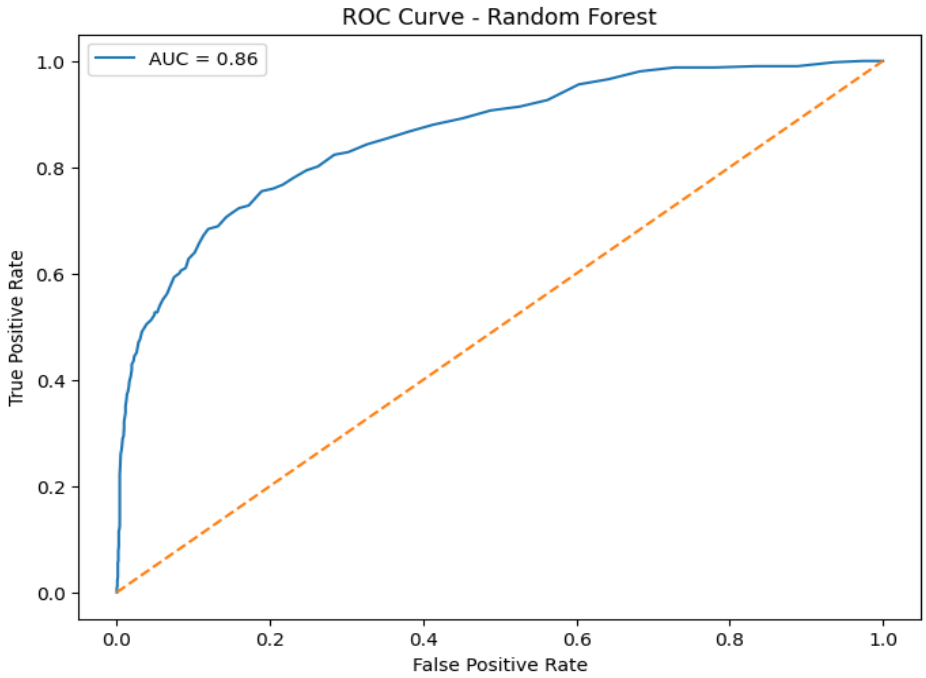
```python
modeloRF_pipeline_A = Pipeline([
    ("processor", preprocessor_A),
    ("model", RandomForestClassifier(
        random_state=42
    ))
])

modeloRF_pipeline_A.fit(X_train, y_train)
prediction_rf = modeloRF_pipeline_A.predict(X_test)
```

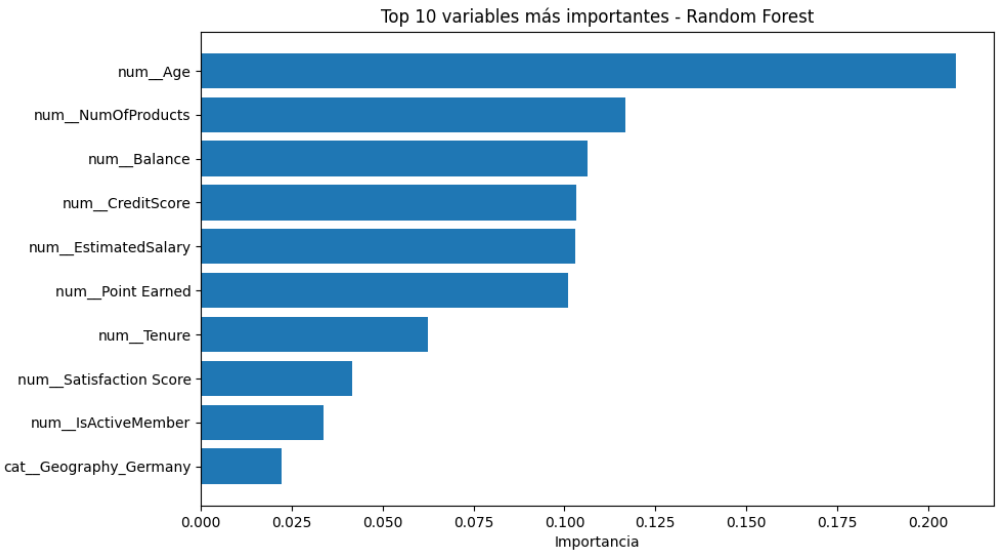
Resultados y visualización

Al desarrollar este modelo, observé una mejora notable. Fue capaz de alcanzar una precisión global del 87%. La matriz de confusión muestra una mejora significativa en comparación con el modelo de regresión logística ya que fue capaz de identificar a más clientes de la clase minoritaria (192 clientes). Desde un punto empresarial esto ya significa un gran avance ya que se reduce el número de falsos negativos. En cuanto al recall, el modelo obtiene un 47% superando nuevamente al otro modelo. Los valores de la curva ROC aumentan ofreciendo un 0.86.

	precision	recall	f1-score	support
0	0.88	0.97	0.92	1592
1	0.81	0.47	0.60	408
accuracy			0.87	2000
macro avg	0.84	0.72	0.76	2000
weighted avg	0.86	0.87	0.86	2000



Posteriormente se graficaron las 10 variables con mayor peso o importancia para el modelo y este fue el resultado:



Pipeline B

Para construir el Pipeline B (avanzado), analicé la cantidad de carencias que yo consideraba que tenía. En primer lugar, me planteé el objetivo de ofrecer una información más rica a los modelos y por ello, creé nuevas variables combinando las que ya existían.

- Age_balance (age * balance): esta variable intenta reflejar la relación que existe entre la edad de una persona y la estabilidad económica.

- Credit_products (CreditScore * NumOfProducts): Esta variable refleja la solvencia de un cliente en ralción con la cantidad de productos contratados en el banco.
- Tenure_Salary (Tenure * EstimatedSalary): Esta variable relaciona la capacidad económica con la antigüedad del cliente.

```
df_pipeline_B["Age_Balance"] = (df_pipeline_B["Age"] * df_pipeline_B["Balance"])
df_pipeline_B["Credit_Products"] = (df_pipeline_B["CreditScore"] * df_pipeline_B["NumOfProducts"])
df_pipeline_B["Tenure_Salary"] = (df_pipeline_B["Tenure"] * df_pipeline_B["EstimatedSalary"])
```

Posteriormente, apliqué transformaciones logarítmicas a las variables con el objetivo de reducir el peso de los valores extremos e igualar las distribuciones. Para las numéricas, apliqué la normalización permitiéndome trabajar con valores similares. Para las categóricas, imputé el valor más frecuente junto con la técnica de One-Hot Encoder para poder ser tratadas como valores numéricos.

```
#Trato los valores extremos
df_pipeline_B["Balance_log"] = np.log1p(df_pipeline_B["Balance"])
df_pipeline_B["EstimatedSalary_log"] = np.log1p(df_pipeline_B["EstimatedSalary"])
```

```
num_transformer_B = Pipeline([("imputer", SimpleImputer(strategy="mean")),
                              ("scaler", StandardScaler())
])
cat_transformer_B = Pipeline([("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])
```

Pipeline B + Regresión Logística

```
modeloLR_pipeline_B = Pipeline([
    ("preprocessor", preprocessor_B),
    ("model", LogisticRegression(
        random_state=42,
        max_iter=5000,
        solver="liblinear",
        class_weight="balanced"
    ))
])
```

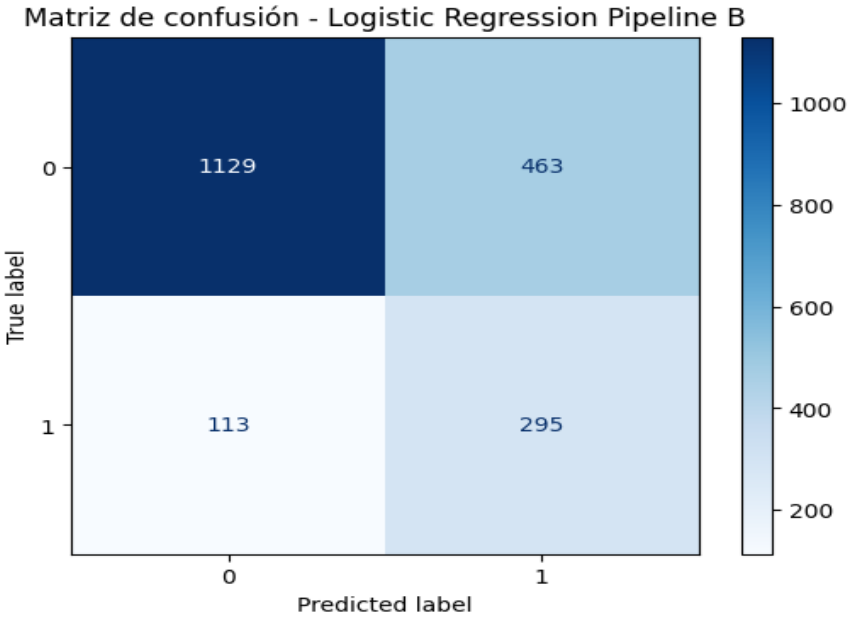
Resultados y visualización

Todo el conjunto de modificaciones del Pipeline B respecto al A mereció la pena. El escalado, balanceo de clases y la creación de nuevas variables ayudaron al modelo a detectar clientes con riesgo de fuga.

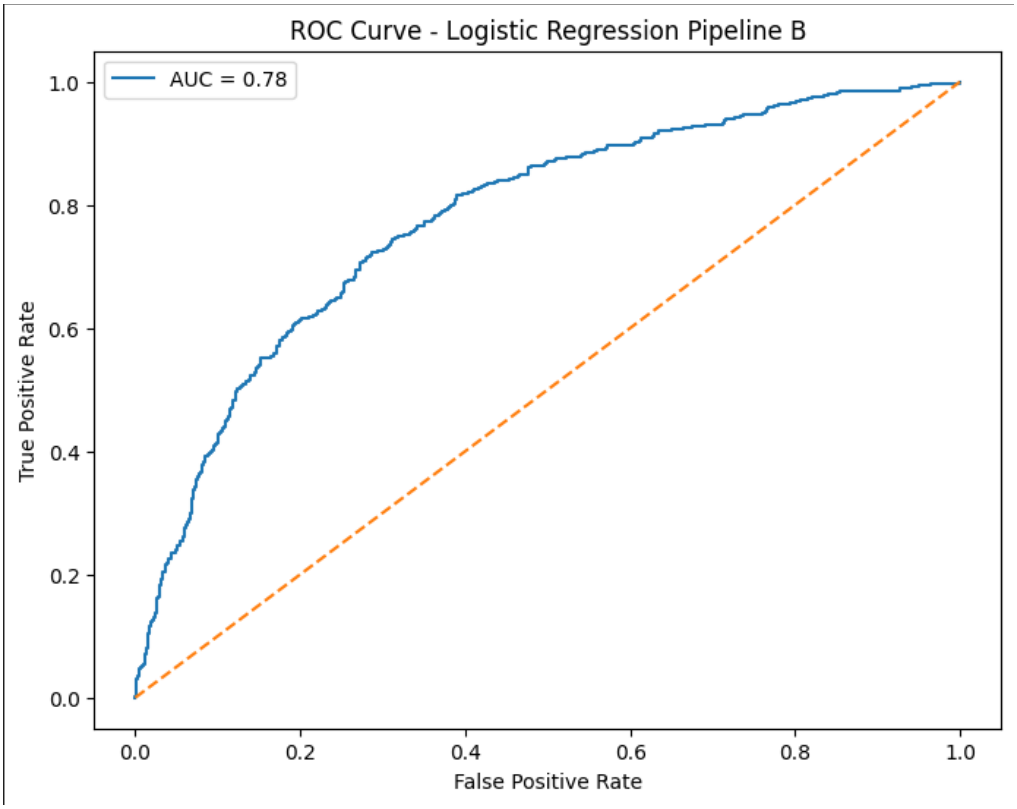
El cambio más notable apareció en la capacidad del modelo para detectar a la clase minoritaria. La combinación del Pipeline A + Regresión Logística captaba apenas un 3% mientras que esta nueva combinación ofrece un 72% (recall). Por otro lado, la matriz de confusión muestra una leve reducción del accuracy del 79% al 71%.

Esto puede entenderse como “normal” ya que el modelo ha dejado de favorecer a esa clase mayoritaria y ha comenzado a detectar más positivos. Además, el f1-score mejoró frente al anterior pipeline ofreciendo un 0.51%.

	precision	recall	f1-score	support
0	0.91	0.71	0.80	1592
1	0.39	0.72	0.51	408
accuracy			0.71	2000
macro avg	0.65	0.72	0.65	2000
weighted avg	0.80	0.71	0.74	2000



La curva Roc muestra una mejora notable a la hora de separar las clases ya que pasa de un 0.67 a un 0.78.



Como conclusión, unas buenas técnicas de feature engineering demuestran que modelos como este, son altamente sensibles y agradecen este tratamiento de los datos.

Pipeline B + Random Forest

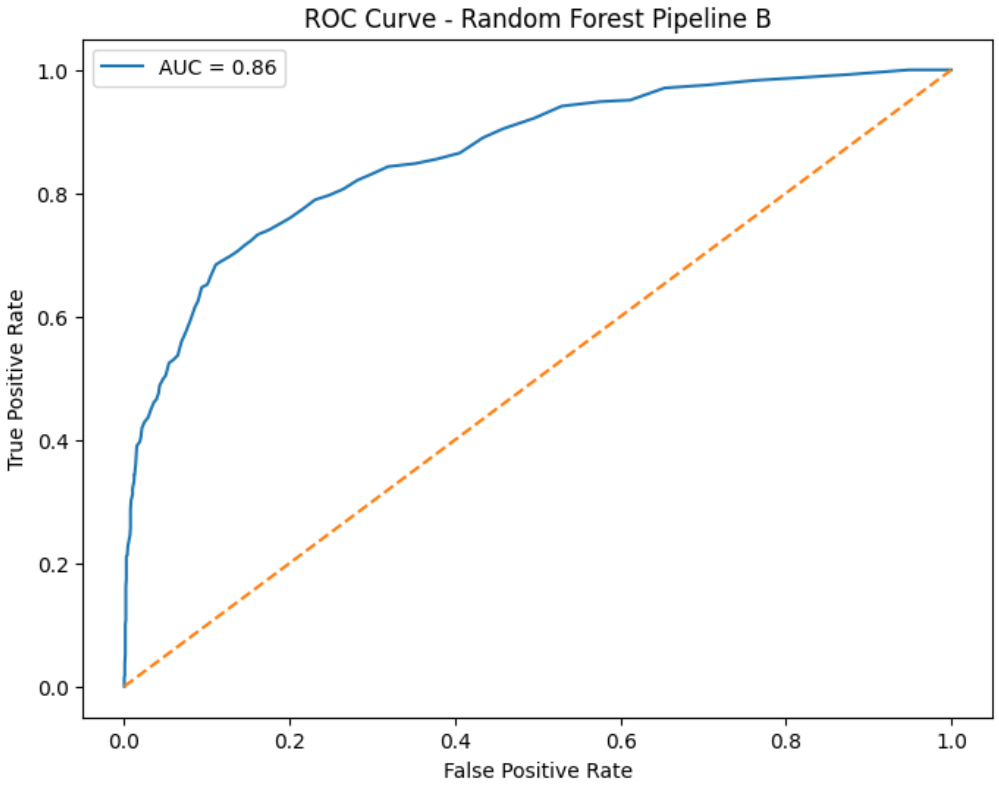
```
modeloRF_pipeline_B = Pipeline([
    ("preprocessor", preprocessor_B),
    ("model", RandomForestClassifier(
        random_state=42,
        class_weight="balanced"
    ))
])
```

Resultados y visualización

Esta ha sido una de las combinaciones que más me ha sorprendido. Esto se debe a que la mejoría no ha sido como esperaba. Se ha comportado de manera muy similar a la combinación del Pipeline A e incluso empeorando algunos valores, además no mejora a Pipeline B + Regresión Logística.

La precisión es buena (86%) pero el recall es mediocre (44%). Los valores de la curva ROC se han mantenido (0.86).

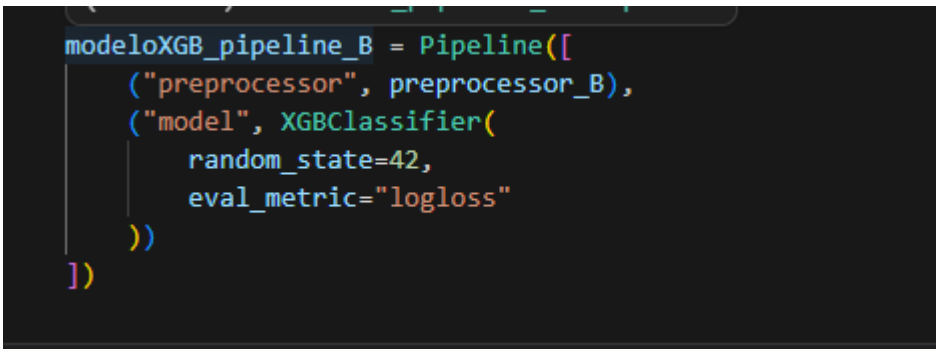
	precision	recall	f1-score	support
0	0.87	0.97	0.92	1592
1	0.79	0.44	0.56	408
accuracy			0.86	2000
macro avg	0.83	0.70	0.74	2000
weighted avg	0.86	0.86	0.85	2000



Como conclusión, entiendo que este comportamiento puede explicarse gracias a la naturaleza del propio modelo. Los árboles de decisión tienen una alta capacidad para detectar patrones no lineales y complejos, todo ello sin necesidad de aplicar técnicas de normalización o transformaciones. Además, este tipo de algoritmos no suelen ser muy sensibles a distribuciones sesgadas.

Pipeline B + XgBoost

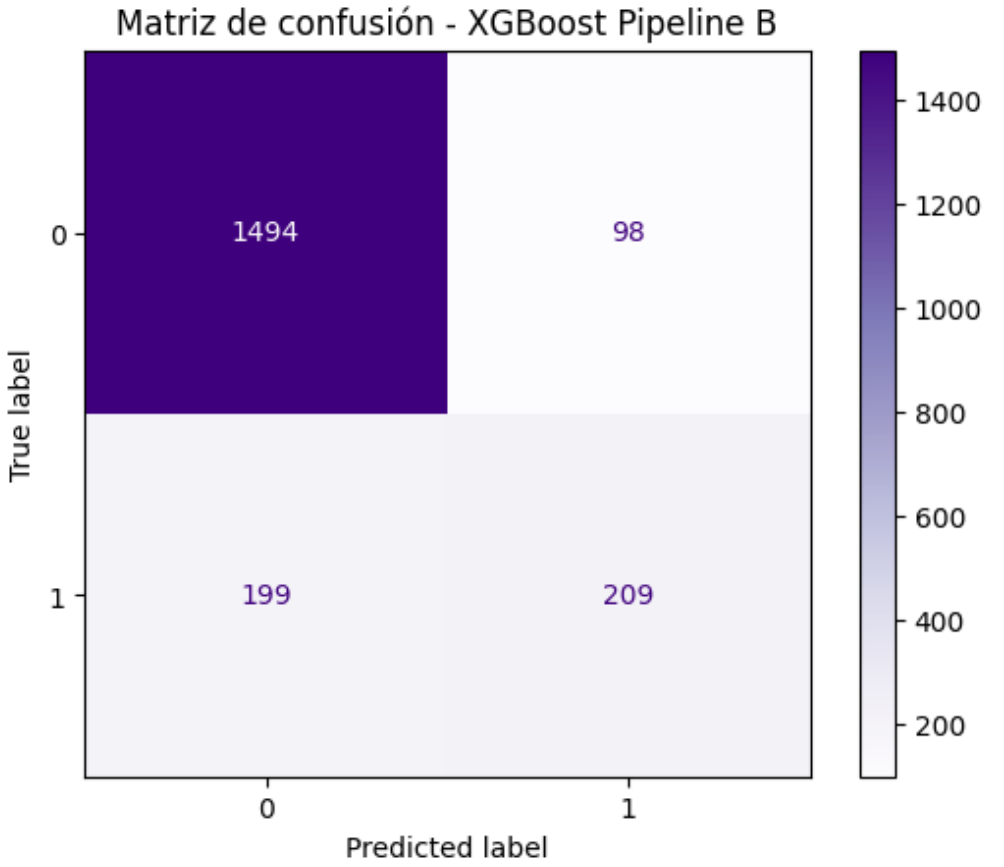
XGBoost (eXtreme Gradient Boosting) es una biblioteca de machine learning distribuida y de código abierto que utiliza árboles de decisión potenciados por gradiente, un algoritmo de boosting de aprendizaje supervisado que utiliza el descenso de gradiente. Es conocido por su velocidad, eficacia y capacidad para escalar bien con grandes conjuntos de datos.



Resultados y visualización

Este modelo ha sido uno de los más equilibrados. Si nos fijamos en las métricas, obtuvo una precisión del 68% pero un 51% de recall. Esto nos indica que aún tiene mucho margen de mejora ya que solo detecta la mitad de los clientes que abandonan el banco. La curva ROC no varió mucho respecto a los otros modelos (0.85). Gracias a la matriz de confusión, podemos ver que ha sido capaz de reducir considerablemente los falsos negativos respecto a los modelos anteriores. Para concluir, el comportamiento del modelo fue estable y equilibrado.

	precision	recall	f1-score	support
0	0.88	0.94	0.91	1592
1	0.68	0.51	0.58	408
accuracy			0.85	2000
macro avg	0.78	0.73	0.75	2000
weighted avg	0.84	0.85	0.84	2000


Matriz de confusión - XGBoost Pipeline B
1494
98
199
209
0
1
True label
0
1
Predicted label
1400
1200
1000
800
600
400
200

Modelo ensamblado: VotingClassifier

Un VotingClassifier consiste en un aprendizaje diseñado para combinar el resultado de múltiples modelos de clasificación. En lugar de tomar decisiones de manera individual, lo que hace es contemplar de manera conjunta la predicción de varios modelos para llegar a un resultado consolidado. (votación)

Hay dos métodos de votación (hard y soft):

- **Hard**: En este modo, cada clasificador base del ensamble realiza su propia predicción para una nueva instancia de datos, emitiendo un voto para una clase específica. El VotingClassifier simplemente cuenta los votos para cada clase. La predicción final del ensamble es la clase que recibe el mayor número de votos. Si hay un empate, el clasificador puede romperlo seleccionando la primera clase en orden ascendente o mediante otro criterio predefinido.
- **Soft**: En lugar de votar por una única clase, cada clasificador proporciona un vector de probabilidades. El VotingClassifier promedia estas probabilidades para cada clase a través de todos los modelos. La predicción final es la clase que tiene la probabilidad promedio más alta. Este enfoque tiene en cuenta la confianza de cada clasificador en su predicción.

Pipeline B + VotingClassifier

En este caso, combiné los 3 modelos que hasta ahora he entrenado, junto con el Pipeline B y junto con la técnica “soft”.

```python
voting_model = VotingClassifier(
    estimators=[
        ("lr", LogisticRegression(
            random_state=42,
            max_iter=5000,
            solver="liblinear",
            class_weight="balanced"
        )),
        ("rf", RandomForestClassifier(
            random_state=42,
            class_weight="balanced"
        )),
        ("xgb", XGBClassifier(
            random_state=42,
            eval_metric="logloss"
        ))
    ],
    voting="soft"
```

```python
modeloVoting_pipeline_B = Pipeline([
    ("preprocessor", preprocessor_B),
    ("model", voting_model)
])
```

✓ 0.0s

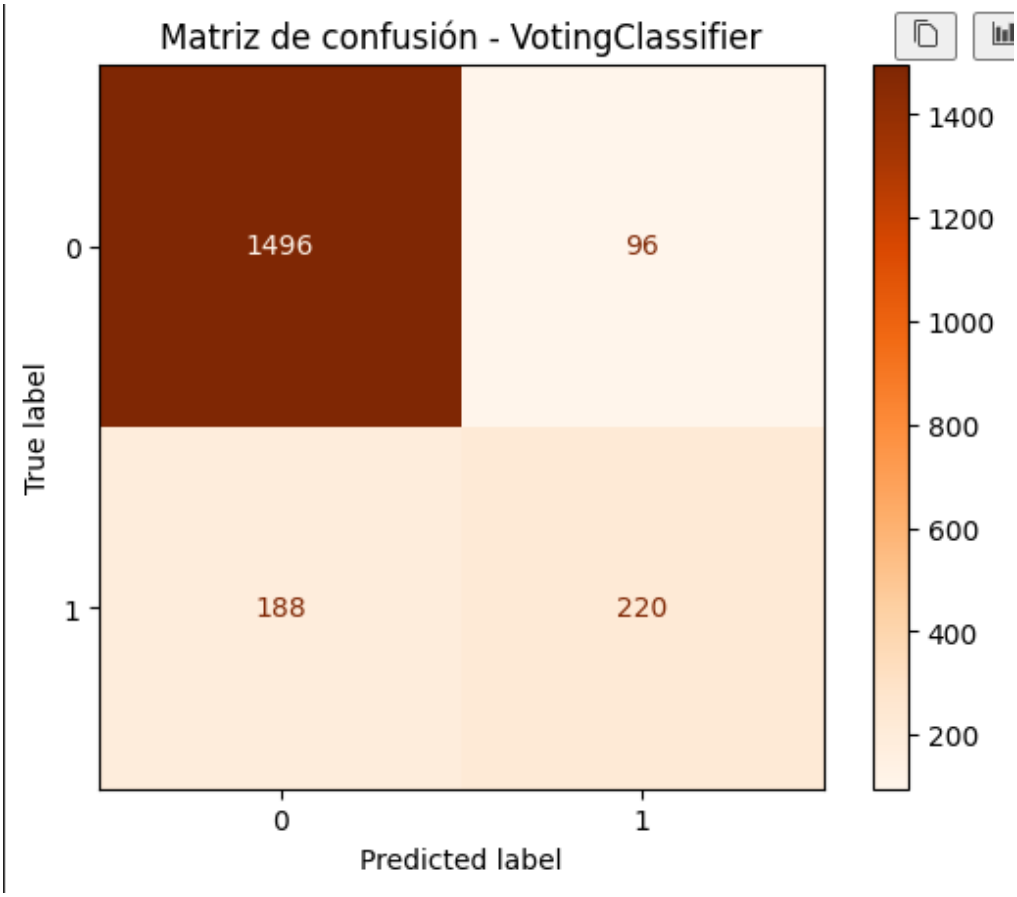
Resultados y visualización

El modelo mostró ser el más equilibrado hasta el momento. Al combinar los 3 modelos de clasificación, consiguió aprovechar los puntos fuertes de cada uno de ellos mostrando estabilidad.

La curva ROC no varió manteniéndose en 0.86, en cambio, F1-score reflejó a la perfección el equilibrio del que hablaba, obteniendo el valor más alto de todos 0.61%. La precisión se mantuvo en 70% y un recall de 54%.

Por último, la matriz de confusión mostró una clara reducción de falsos negativos, permitiendo identificar a más clientes con riesgo de abandono del banco.

	precision	recall	f1-score	support
0	0.89	0.94	0.91	1592
1	0.70	0.54	0.61	408
accuracy			0.86	2000
macro avg	0.79	0.74	0.76	2000
weighted avg	0.85	0.86	0.85	2000

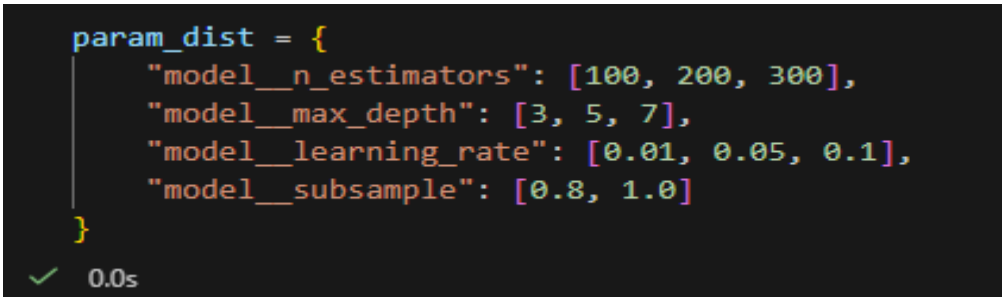


Tuneado de Hiperparámetros para XGBOOST

Deposité toda mi confianza en el modelo XGBoost y por ello, decidí realizar una optimización de los hiperparámetros. La finalidad de este proceso fue aplicar una configuración adecuada del modelo que

permitiera mejorar la precisión y la capacidad de detección de clientes que abandonan el banco.

Entre estas mejoras, incluí el número de árboles(n_estimators), la profundidad(max_depth), tasa de aprendizaje(learning_rate) y el porcentaje de muestras usadas en cada iteración(subsample). Posteriormente se unificó con el pipeline B y, al no disponer de un conjunto de datos nuevos, decidí aplicar la validación cruzada gracias a RandomizedSearchCV. Se configuró una búsqueda aleatoria de 10 combinaciones diferentes de hiperparámetros (n_iter=10) junto con validación cruzada de 3 particiones (cv=3). Además, se seleccionó el F1-score como métrica principal de evaluación (scoring="f1"), ya que el problema presentaba un importante desbalanceo entre clases y esta métrica ofrecía una visión más equilibrada del comportamiento del modelo.



```
random_search = RandomizedSearchCV(
    estimator=pipeline_tuning,
    param_distributions=param_dist,
    n_iter=10,
    cv=3,
    scoring="f1",
    verbose=2,
    random_state=42,
    n_jobs=-1
)
```

0.0s

El objetivo fue entrenar el modelo ensamblado junto con las nuevas configuraciones de XGBoost, pero, aun así, entrené el modelo de manera individual sin aplicar el ensamblado y este fue el resultado:

	precision	recall	f1-score	support
0	0.88	0.95	0.92	1592
1	0.74	0.51	0.60	408
accuracy			0.86	2000
macro avg	0.81	0.73	0.76	2000
weighted avg	0.85	0.86	0.85	2000

Pipeline B + VotingClassifier + XGBoost optimizado

```
voting_model_tuned = VotingClassifier(
    estimators=[
        ("lr", LogisticRegression(
            random_state=42,
            max_iter=5000,
            solver="liblinear",
            class_weight="balanced"
        )),
        ("rf", RandomForestClassifier(
            random_state=42,
            class_weight="balanced"
        )),
        ("xgb", xgb_tuned_for_voting)
    ],
    voting="soft"
)
```

Resultados y visualización

Finalmente, pude entrenar el último modelo que albergaba todas las optimizaciones para ver si era capaz de mejorar su rendimiento.

Este, sí fue el que más equilibrio mostró ya que alcanzó una precisión del 71%. Asimismo, el recall aumentó al 56% dando unos de los mejores valores. Como métrica de valoración general, F1-score marcaba 0.62 convirtiéndose en el mejor resultado. La curva ROC se mantuvo en 0.86.

	precision	recall	f1-score	support
0	0.89	0.94	0.92	1592
1	0.71	0.56	0.62	408
accuracy			0.86	2000
macro avg	0.80	0.75	0.77	2000
weighted avg	0.86	0.86	0.86	2000

Elección del modelo

	Modelo	Accuracy	Precision	Recall	F1-Score	ROC-AUC
0	Logistic Regression A	0.79	0.41	0.03	0.06	0.67
1	Random Forest A	0.87	0.81	0.47	0.60	0.86
2	Logistic Regression B	0.71	0.39	0.72	0.51	0.78
3	Random Forest B	0.86	0.79	0.44	0.56	0.86
4	XGBoost B	0.85	0.68	0.51	0.58	0.85
5	XGBoost Tuneado	0.86	0.74	0.51	0.60	0.85
6	VotingClassifier	0.86	0.70	0.54	0.61	0.86
7	VotingClassifier Tuneado	0.86	0.71	0.56	0.62	0.86

Tras valorar todos los modelos y métricas decidí que el modelo ensamblado VotinClassifier con la optimización, era el más adecuado para este caso de uso. Había otros modelos interesantes que

mostraron resultados realmente buenos, pero, a causa del desbalanceo entre clases, no sólo se atendió al accuracy y se priorizaron otras métricas como recall, F1-score y la curva ROC.

El modelo de Regresión Logística B obtuvo el mejor resultado de recall, en cambio, presentó la precisión más baja de todas. Esto iba a desencadenar en una detección de muchos falsos positivos, y eso no conviene a la empresa.

Por otro lado, Random Forest B a priori parece ser el modelo con mejores resultados. El hecho de que lo haya descartado se debe a que mostró un recall bastante flojo por lo que iba a detectar menos casos reales de churn.

Finalmente, atendiendo al mejor F1-score, recall aceptable, buen ROC y una precisión equilibrada, opté por escoger a VotingClassifier + XGBoost tuneado como mejor modelo para detectar clientes con posible riesgo de fuga del banco.

Conclusión

A lo largo de este proyecto, he realizado el ciclo de vida completo del desarrollo de una solución tecnológica en el ámbito del Machine

Learnig. Desde la extracción de los datos, feature egineering, entrenamiento y validación y evaluación.

Teniendo en cuenta las métricas aprendidas a lo largo del máster, he sido capaz de observar con criterio y saber analizar los resultados de los modelos. Como resultado, he obtenido que el modelo VotingClassifier + XGBoost tuneado es el más equilibrado para ser aplicado a este caso de uso.

Como posibles mejoras, considero que estría bien crear alguna especie de interfaz para el equipo del banco. Posteriormente, introducir un sistema automático para que, en el momento de detectar un riesgo de fuga, se envíen ofertas o recomendaciones mejorando la calidad del servicio y su experiencia.

Finalmente, no sólo ha significado un trabajo más, si no que me ha servido para poner a prueba todo lo aprendido durante el curso. Así como observar la capacidad extraordinaria y potencia que tiene el Machine Learning cuando se trabaja con datos.

Bibliografía

- **Scikit-Learn**

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, É. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

- **Dataset**

  Kaggle. (s.f.). *Bank Customer Churn Prediction Dataset*. Recuperado de: https://www.kaggle.com/code/bryandelgadoquijano/de-datos-a-decisiones-churn-bancario

- **XGBoost**

  Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-794. https://doi.org/10.1145/2939672.2939785

- OpenAI. (2025). *ChatGPT* (modelo GPT-5.5) [Large language model]. https://chatgpt.com/